

ABE 721

Agricultural Systems Modelling and Simulation

Module 3: Qualitative and Quantitative Simulation Paradigms

Prepared by:

Kadeghe Fue, PhD, P.Eng(T)

Electronics and Precision Agriculture Laboratory (EPAL)

Sokoine University of Agriculture (SUA)

Morogoro, Tanzania

Contents

3 Qualitative and Quantitative Simulation **3**

3.1 3.1 Simulation Paradigms 3

3.1.1 Qualitative vs. Quantitative Simulation 4

3.1.2 Continuous vs. Discrete-Event Simulation 5

3.2 3.2 Advanced Numerical Techniques for Differential Equations 7

3.2.1 The Mathematical Foundation: Taylor Series Expansion 7

3.2.2 First-Order Methods: Forward Euler 7

3.2.3 Second-Order Methods: Heun's Method (Modified Euler) . . . 8

3.2.4 Fourth-Order Runge-Kutta Method (RK4) 8

3.2.5 Numerical Solutions for PDEs: Finite Difference Method (FDM) 11

3.2.6 Finite Element Analysis (FEA/FEM) 13

3.3 3.3 Parameter Estimation and Inverse Modelling 15

3.3.1 The Objective Function Formulation 15

3.3.2 Gradient-Based Optimization Techniques 16

3.3.3 Soft Computing and Metaheuristic Algorithms 17

Chapter 3

Qualitative and Quantitative Simulation

In Module 2, we formulated agricultural realities into rigorous mathematical models, translating physical phenomena into differential equations and algebraic systems. However, formulating a theoretical model represents only the first half of the engineering challenge. Because the vast majority of real-world agricultural models are highly non-linear, they lack simple analytical (exact) solutions.

Module 3 focuses on the *execution* of these models. We explore the computational simulation paradigms used to advance time in a digital environment, the complex numerical integration techniques required to solve differential equations computationally, and the advanced optimization algorithms required to estimate and calibrate model parameters using real-world data (Tayfur, 2014).

3.1 3.1 Simulation Paradigms

Simulation is the imitation of the operation of a real-world process or system over time. The computational paradigm chosen by the engineer completely dictates how the computer memory advances time, tracks the system's state variables, and handles uncertainty.

3.1.1 Qualitative vs. Quantitative Simulation

Quantitative Simulation

Quantitative models rely on strict, deterministic numerical data, hard boundaries, and explicit mathematical equations based on the laws of physics and biology. They attempt to yield exact, numerically precise forecasts.

- *Mathematical Foundation:* Differential calculus, linear algebra, and strict stochastic probability distributions.
- *Agricultural Use Case:* Precise engineering design. For example, using the Navier-Stokes equations to calculate the exact optimal PVC pipe diameter and pump horsepower required for a drip irrigation network to maintain a pressure of exactly 200 kPa.

Qualitative Simulation and Soft Computing

Agricultural systems frequently involve socio-technical and environmental variables that are inherently difficult to measure precisely (e.g., “farmer satisfaction,” “soil health,” “crop visual stress,” or “market volatility”). Qualitative simulation deals with trends, behaviors, and fuzzy boundaries rather than exact numbers.

As heavily referenced in modern literature (Tayfur, 2014), **Fuzzy Logic** is the primary mathematical tool for qualitative simulation. Unlike Boolean logic where a statement is either true (1) or false (0), fuzzy logic utilizes **Membership Functions** (μ) where an element can partially belong to a set (e.g., 0.75 true).

Let X be the universe of discourse for soil moisture (e.g., 0 to 40% volumetric water content). A fuzzy set A (e.g., ”Dry Soil”) is defined by its membership function $\mu_A(x)$, mapping every $x \in X$ to a membership degree between 0 and 1.

$$A = \{(x, \mu_A(x)) | x \in X\} \tag{3.1}$$

By defining fuzzy rules (e.g., *”IF Soil is Dry AND Temperature is Hot, THEN Irrigation is High”*), engineers can simulate qualitative expert human decision-making pro-

cesses for automated greenhouse controllers without requiring strict differential equations.

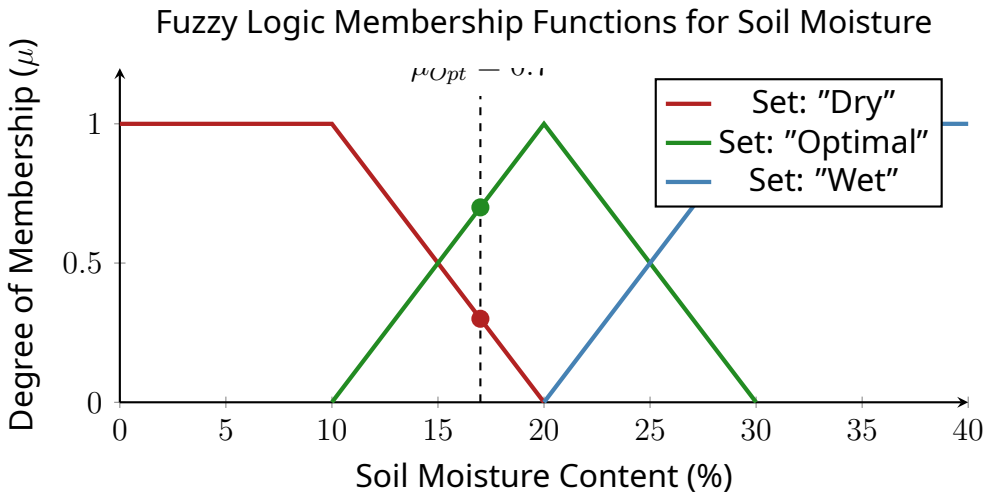


Figure 3.1: Qualitative simulation using Fuzzy Logic. A strict quantitative model would force 17% moisture into a single rigid category. Fuzzy logic acknowledges that 17% belongs simultaneously to the "Dry" set (degree 0.3) and the "Optimal" set (degree 0.7), allowing for smoother, human-like algorithmic decision making.

3.1.2 Continuous vs. Discrete-Event Simulation

Beyond data types, the fundamental architecture of a computational model is determined by how it advances its internal clock.

Continuous Simulation

In continuous simulation, state variables change smoothly and continuously over time. The computer clock is advanced in tiny, uniform, rigid increments (Δt).

- *Mathematics:* Governed strictly by Ordinary Differential Equations (ODEs) and Partial Differential Equations (PDEs).
- *Simulation Engine:* The model computes the derivatives at t, calculates the new state at $t + \Delta t$, updates the clock, and repeats. If nothing happens for 10 hours, the computer still calculates every single micro-step between those hours.

Discrete-Event Simulation (DES)

In DES, the system state only changes at discrete, specific points in time triggered by scheduled ”events.” Between these events, the system state is assumed to remain perfectly constant.

- *Mathematics:* Governed by Queuing Theory, probability arrays, and logical state-machines.
- *Simulation Engine:* The computer maintains an ”Event List.” The clock does not tick uniformly; it instantly jumps to the exact time of the next scheduled event, bypassing periods of inactivity entirely.
- *Agricultural Example:* Simulating a sugar mill receiving queue. The system is modeled as an $M/M/1$ queue (Markovian arrival, Markovian service time, 1 server). The events are: *Tractor Arrives at* $t = 2.1 \rightarrow$ *Weighing Begins* $\rightarrow$ *Weighing Ends at* $t = 2.5 \rightarrow$ *Unloading Begins.*

Cellular Automata and Agent-Based Modelling (ABM)

A growing paradigm in spatial agricultural modelling. In Agent-Based Modelling, the system is modeled as a collection of autonomous decision-making entities called ”agents” (e.g., individual insects, or individual farmers). Each agent assesses its situation and makes decisions based on a set of rules. ABM is heavily used to simulate the spread of plant diseases across regions or the adoption of new farming policies.

3.2 3.2 Advanced Numerical Techniques for Differential Equations

As established in previous modules, continuous agricultural models are formulated as differential equations. For instance, the general Initial Value Problem (IVP) is defined as:

$$\frac{dy}{dt} = f(t, y) \quad \text{with initial condition} \quad y(t_0) = y_0 \tag{3.2}$$

Because analytical solutions (finding an exact algebraic equation for $y(t)$ via integration) are mathematically impossible for highly complex, non-linear farm systems, we must use **Numerical Integration**. Numerical methods step through time, calculating approximate values of y at discrete time steps: $t_1, t_2, ..., t_n$ where $t_{n+1} = t_n + h$. Here, h is the step size (Δt).

3.2.1 The Mathematical Foundation: Taylor Series Expansion

All numerical integration methods, without exception, are mathematically derived from the Taylor series. If a state function $y(t)$ is continuous and differentiable, its future value at the next time step t_{n+1} can be perfectly projected from its current state $y(t_n)$ and the infinite sum of its derivatives:

$$y(t_{n+1}) = y(t_n) + hy'(t_n) + \frac{h^2}{2!}y''(t_n) + \frac{h^3}{3!}y'''(t_n) + \cdots + O(h^p) \tag{3.3}$$

Where $O(h^p)$ represents the local truncation error resulting from ignoring higher-order terms.

3.2.2 First-Order Methods: Forward Euler

The simplest numerical technique is the explicit Forward Euler method. It simply truncates the Taylor series after the first derivative, completely ignoring the acceleration (y'') and jerk (y''') of the system.

Derivation:

$$y_{n+1} = y_n + hf(t_n, y_n) \tag{3.4}$$

Truncation Error Analysis: Because we dropped the $\frac{h^2}{2!}y''(t_n)$ term, the *Local Truncation Error* (the error introduced in a single step) is proportional to $O(h^2)$. However, errors accumulate over the entire simulation. The *Global Truncation Error* for the Euler method is $O(h)$. This designates Euler as a "First-Order" method. If you halve the time step ($h/2$), the global error is halved, but the computational time doubles. Euler's method is inherently inaccurate for non-linear curves because it assumes the slope remains perfectly constant over the entire step h.

3.2.3 Second-Order Methods: Heun's Method (Modified Euler)

To improve upon Euler without calculating complex second derivatives, we use predictor-corrector methods. Heun's method averages the slope at the beginning of the step and the estimated slope at the end of the step.

Step 1 (Predictor): Calculate the standard Euler prediction for the end of the step.

$$\tilde{y}_{n+1} = y_n + hf(t_n, y_n) \tag{3.5}$$

Step 2 (Corrector): Calculate the final state using the average of the initial slope and the predicted final slope.

$$y_{n+1} = y_n + \frac{h}{2}\left[f(t_n, y_n) + f(t_{n+1}, \tilde{y}_{n+1})\right] \tag{3.6}$$

By averaging the slopes, Heun's method achieves a Global Truncation Error of $O(h^2)$, making it a Second-Order method.

3.2.4 Fourth-Order Runge-Kutta Method (RK4)

The Runge-Kutta family of methods expands the predictor-corrector logic by evaluating the first derivative $f(t, y)$ at multiple strategic points within the step h and computing a highly weighted average slope.

The **Fourth-Order Runge-Kutta (RK4)** is the absolute gold standard and workhorse of continuous agricultural systems simulation. It is exceptionally stable and features a global truncation error of $O(h^4)$. If you halve the time step, the error drops

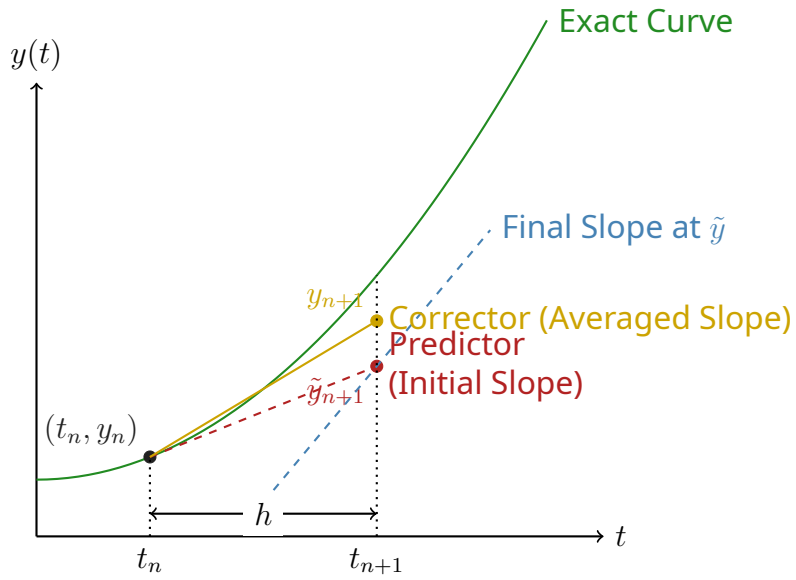


Figure 3.2: Geometric Interpretation of Heun's Method. The raw Euler predictor significantly undershoots the true curve. By evaluating the slope at the predicted point and averaging it with the initial slope, the corrector step y_{n+1} achieves a much closer approximation to the exact curve.

by a factor of 16.

The Mathematical RK4 Algorithm: To advance the system state from t_n to t_{n+1}, calculate four intermediate slope evaluations (k-values):

$$k_1 = f(t_n, y_n) \quad \text{(Slope at the beginning of the interval)} \tag{3.7}$$

$$k_2 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right) \quad \text{(Slope at midpoint, using } k_1 \text{ to project)} \tag{3.8}$$

$$k_3 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right) \quad \text{(Slope at midpoint, using } k_2 \text{ to project)} \tag{3.9}$$

$$k_4 = f(t_n + h, y_n + hk_3) \quad \text{(Slope at the end of the interval, using } k_3 \text{ to project)} \tag{3.10}$$

Finally, calculate the new state using a weighted average slope, heavily weighting the midpoint estimates (k_2 and k_3):

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \tag{3.11}$$

Advanced Application Note: In modern engineering software (like MATLAB's 'ode45' or Python's 'scipy.integrate.solve_ivp'), solvers use adaptive step-size Runge-Kutta-

Fehlberg algorithms (RK45). The solver calculates the step using both an RK4 and an RK5 method. The difference between the two results is used as an estimate of the local error. If the error is too high (e.g., during a sudden rainfall event changing soil moisture rapidly), the algorithm automatically shrinks h. If the error is very low (e.g., dormant winter conditions), the algorithm expands h to save processing memory.

3.2.5 Numerical Solutions for PDEs: Finite Difference Method (FDM)

While ODEs model variables with respect to *time* alone (creating ”Lumped” models), Partial Differential Equations (PDEs) model variables with respect to both *time and space* (creating ”Distributed” models).

Consider the 1D Heat Equation (or Diffusion Equation), representing heat moving through a greenhouse soil profile:

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2} \tag{3.12}$$

Where $u(x,t)$ is temperature, and α is thermal diffusivity. To solve this computationally, we utilize the **Finite Difference Method (FDM)**. FDM replaces continuous partial calculus derivatives with discrete algebraic difference approximations based on a space-time grid matrix.

Discretization of the Grid Domain

We discretize the soil depth (x) into spatial nodes separated by Δx, and time (t) into steps separated by Δt. Let u_i^n denote the temperature at spatial node i and time step n.

Using Taylor series expansions, we approximate the temporal derivative using a Forward Difference and the spatial derivative using a Central Difference:

$$\frac{\partial u}{\partial t} \approx \frac{u_i^{n+1} - u_i^n}{\Delta t} \tag{3.13}$$

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{(\Delta x)^2} \tag{3.14}$$

The Explicit FDM Scheme and CFL Stability

Substituting the approximations into the PDE yields the algebraic **Explicit Scheme**:

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \alpha \left(\frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{(\Delta x)^2} \right) \tag{3.15}$$

Solving for the unknown future state u_i^{n+1} isolates it on the left side:

$$u_i^{n+1} = u_i^n + \frac{\alpha \Delta t}{(\Delta x)^2} \left(u_{i+1}^n - 2u_i^n + u_{i-1}^n \right) \tag{3.16}$$

While computationally simple, Explicit FDM suffers from severe instability if time steps are too large. Through Von Neumann stability analysis, engineers derived the **Courant-Friedrichs-Lewy (CFL) Condition**. For the 1D diffusion equation, the explicit scheme is only stable if:

$$CFL = \frac{\alpha \Delta t}{(\Delta x)^2} \leq 0.5 \tag{3.17}$$

If the spatial resolution Δx is halved to increase accuracy, Δt must be divided by 4 to prevent the simulation from exploding to infinity, resulting in massive computational costs.

The Crank-Nicolson Method

To bypass the strict CFL stability limit, modern agricultural models utilize the Implicit Crank-Nicolson method. It averages the spatial derivative across both the current time step n and the future time step $n+1$:

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \frac{\alpha}{2} \left[\frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{(\Delta x)^2} + \frac{u_{i+1}^{n+1} - 2u_i^{n+1} + u_{i-1}^{n+1}}{(\Delta x)^2} \right] \tag{3.18}$$

This method is *unconditionally stable* (CFL limits do not apply) and achieves Second-Order accuracy in both time and space. However, because u^{n+1} appears on both sides of the equation across multiple nodes, it cannot be solved simply node-by-node. It requires formulating a massive tridiagonal matrix system ($\mathbf{A}u^{n+1} = \mathbf{B}u^n$) and solving it using linear algebra techniques like the Thomas Algorithm.

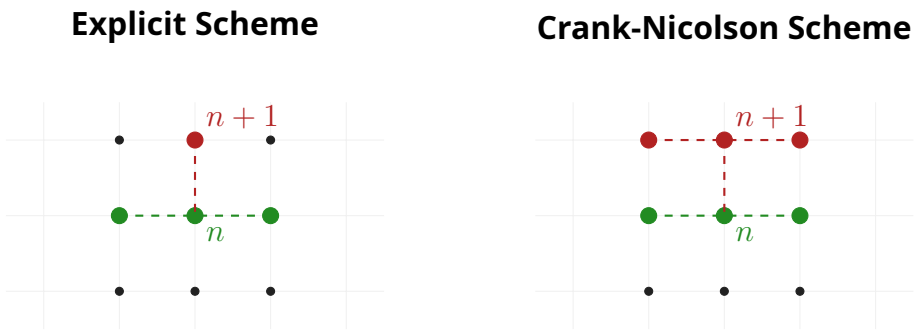


Figure 3.3: Computational stencils for Finite Difference Methods. Green nodes represent known data at the current time step; Red nodes are the unknown future states. The Explicit method calculates the future based solely on the present. The Crank-Nicolson method creates a coupled system where the future state of node i simultaneously depends on the unknown future states of its spatial neighbors ($i-1$ and $i+1$).

3.2.6 Finite Element Analysis (FEA/FEM)

While FDM is excellent for fluid and thermodynamic modelling over simple rectangular grid domains, it fails catastrophically when applied to the complex, curved 3D geometries typical in structural machine design (e.g., a tractor chassis or a curved moldboard plow).

As referenced by Mukhopadhyay (2000) and Krishnamoorty (2001) in the syllabus, the **Finite Element Method (FEM)** is the standard for mechanical stress analysis.

The Core Concept of FEM

Instead of laying a grid over the domain, FEM geometrically divides the continuous 3D solid CAD model into thousands of discrete, interconnected geometric subvolumes (typically tetrahedrons or hexahedrons) called **Finite Elements**. These elements are connected at points called **Nodes**.

Mathematical Formulation

The physical behavior of a continuous solid under stress is governed by a highly complex PDE (Navier-Cauchy equations). FEM utilizes the *Weak Formulation* and the *Principle of Virtual Work* to convert this PDE into a massive system of simultaneous algebraic equations.

For static structural analysis, the fundamental governing matrix equation is:

$$[\mathbf{K}]\{\mathbf{U}\} = \{\mathbf{F}\} \tag{3.19}$$

Where:

- $[\mathbf{K}]$ is the **Global Stiffness Matrix**. It is mathematically assembled by calculating the local stiffness matrix of every individual geometric element based on the material's Young's Modulus (E) and Poisson's ratio (ν).
- $\{\mathbf{U}\}$ is the **Nodal Displacement Vector** (the unknown variables). It represents how much every single node in the machine bends or moves.
- $\{\mathbf{F}\}$ is the **Applied Force Vector** (the known loads and boundary conditions).

Once the computer algebraically solves the massive inverted matrix system $\{\mathbf{U}\} = [\mathbf{K}]^{-1}\{\mathbf{F}\}$, it calculates the internal stress and strain utilizing Hooke's Law ($\sigma = E\epsilon$). If the calculated stress exceeds the steel's yield strength, the engineer knows the machinery part will physically break during operation.

3.3 3.3 Parameter Estimation and Inverse Modelling

A mechanistically perfect mathematical model consisting of advanced PDEs, solved flawlessly with a Crank-Nicolson algorithm, will still produce entirely useless, highly inaccurate predictions if the **parameters** within the equations are incorrect.

Parameters represent the fixed properties of the system (e.g., the hydraulic conductivity of soil K_s, the specific heat capacity of a greenhouse canopy, or the biological radiation conversion efficiency of a maize cultivar). Parameter estimation (often referred to as model calibration or inverse modelling) is the rigorous mathematical process of computationally finding the optimal set of parameters **p** such that the model's simulated outputs closely match real-world historical observed data.

3.3.1 The Objective Function Formulation

Parameter estimation is fundamentally framed as a mathematical **Optimization Problem**. The engineer must define a cost function (or objective function) that quantifies the total "error" between the simulation predictions and reality.

Let $y_{obs}(t_i)$ be the actual observed data point at time t_i, and $y_{sim}(t_i, \mathbf{p})$ be the model's simulated output generated precisely at time t_i using a specific parameter set **p**.

The universally applied objective function is the **Sum of Squared Errors (SSE)**:

$$J(\mathbf{p}) = \sum_{i=1}^{N} [y_{obs}(t_i) - y_{sim}(t_i, \mathbf{p})]^2 \tag{3.20}$$

The ultimate mathematical goal of parameter estimation is to find the singular optimal parameter vector $\mathbf{p}^*$ that globally minimizes the objective function $J(\mathbf{p})$:

$$\mathbf{p}^* = \arg\min_{\mathbf{p}} J(\mathbf{p}) \tag{3.21}$$

3.3.2 Gradient-Based Optimization Techniques

How does a computer systematically find the absolute minimum of this complex, multidimensional, non-linear error surface $J(\mathbf{p})$?

The Levenberg-Marquardt Algorithm (LMA)

Because agricultural mechanistic models are highly non-linear, analytical algebraic solutions (like Ordinary Least Squares) are impossible. We must utilize iterative numerical gradient descent solvers. The Levenberg-Marquardt Algorithm (LMA) is the gold-standard algorithm for Non-Linear Least Squares (NLLS) curve fitting.

LMA bridges the gap between the steepest descent method (stable but slow) and the Gauss-Newton algorithm (fast but unstable). It iteratively updates the parameter guess:

$$\mathbf{p}_{k+1} = \mathbf{p}_k - \left[\mathbf{J}^T\mathbf{J} + \lambda \text{diag}(\mathbf{J}^T\mathbf{J})\right]^{-1}\mathbf{J}^T\mathbf{e}(\mathbf{p}_k) \tag{3.22}$$

Where:

- $\mathbf{J}$ is the Jacobian matrix containing the partial derivatives of the simulated outputs with respect to the parameters. It points the algorithm "downhill" towards lower error.
- $\mathbf{e}(\mathbf{p}_k)$ is the residual error vector.
- λ is the algorithmic damping parameter. When the current guess is far from the minimum, λ is large, and LMA acts like safe, slow gradient descent. When the error is small and close to the minimum, λ shrinks towards zero, and LMA acts like the lightning-fast Gauss-Newton method to pinpoint the exact bottom of the error curve.

3.3.3 Soft Computing and Metaheuristic Algorithms

A critical, often fatal limitation of gradient-based algorithms like LMA is the risk of getting trapped in a *local minimum*. The algorithm might find a parameter set that looks mathematically optimal locally (the bottom of a small valley), but fails to find the true *global optimum* (the deepest valley on the entire map).

For highly complex agricultural systems possessing dozens of interlinked parameters (e.g., calibrating the DSSAT or APSIM crop growth suites), computing the exact gradient Jacobian **J** is computationally impossible. To solve this, researchers deploy **Soft Computing and Metaheuristic Algorithms**, which are heavily inspired by biology and nature (Tayfur, 2014).

1. Genetic Algorithms (GA)

Genetic Algorithms are heuristic search algorithms inspired by Charles Darwin's theory of natural evolution. They do not use gradients or derivatives at all; they use probabilistic natural selection.

The GA Iterative Cycle:

1. **Initialization:** Generate a massive random "population" of thousands of different parameter sets ($\mathbf{p}_1, \mathbf{p}_2, \dots$). Each parameter set is encoded as a "Chromosome".
2. **Fitness Evaluation:** Run the agricultural simulation model for every single chromosome. Calculate the SSE error $J(\mathbf{p})$ for each. The "Fitness" score is the inverse of the error ($1/J$).
3. **Selection:** Using a "Roulette Wheel" or "Tournament" probability selection method, choose the chromosomes with the highest fitness to become "Parents". The parameter sets that generated massive errors are killed off and removed from the pool.
4. **Crossover (Mating):** Combine the numerical values of two successful parent parameter sets to generate a new "Child" parameter set that inherits traits from both.

5. **Mutation:** Randomly tweak a small percentage of parameter values in the children. This biological mutation prevents the population from stagnating and forces the algorithm to explore entirely new areas of the parameter map.

6. **Iteration:** The new children become the population for the next generation. This repeats for hundreds of generations until the parameter sets converge on the true global minimum error.

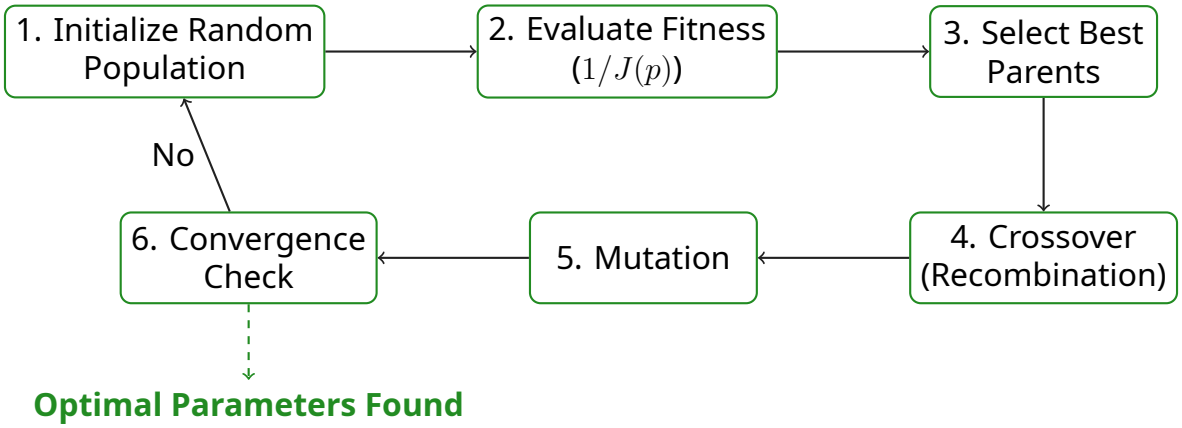


Figure 3.4: The iterative evolutionary cycle of a Genetic Algorithm (GA) used for complex agricultural parameter estimation without requiring mathematical derivatives.

2. Particle Swarm Optimization (PSO)

While GA simulates evolution, Particle Swarm Optimization mimics the social kinematics of a flock of birds or a school of fish searching an area for food (the lowest error point).

Instead of chromosomes, PSO maintains a swarm of "Particles". Each particle represents a specific parameter set $\mathbf{p}_i$, moving through the multidimensional search space with a specific velocity $\mathbf{v}_i$. The velocity of each particle is dynamically updated every iteration based on its own personal best historical memory (P_{best}) and the swarm's collective global best discovery (G_{best}).

The velocity update equation for a particle is:

$$\mathbf{v}_i^{(t+1)} = w\mathbf{v}_i^{(t)} + c_1 r_1(\mathbf{P}_{best,i} - \mathbf{p}_i^{(t)}) + c_2 r_2(\mathbf{G}_{best} - \mathbf{p}_i^{(t)}) \tag{3.23}$$

Where w is the inertia weight, c_1 is the cognitive (personal) parameter, c_2 is the

social (swarm) parameter, and r_1, r_2 are random numbers between 0 and 1. The particle's new parameter position is then simply updated by kinematics:

$$\mathbf{p}_i^{(t+1)} = \mathbf{p}_i^{(t)} + \mathbf{v}_i^{(t+1)} \tag{3.24}$$

PSO is often faster to converge than Genetic Algorithms in continuous mathematical spaces, making it highly preferred for calibrating hydrological and soil models.

End of Module 3.